Alex Picard

Brookline, MA · (207) 551-5430 · am.picard03@gmail.com

https://www.linkedin.com/in/alexpicard0/ | https://github.com/Alexm-picard | https://alexpicard.info

Software engineer who builds and operates production AI/ML systems: shipped a full-stack RAG platform and a self-hosted ML serving platform (p99 34 ms at a verified 300 req/s), with an ML research background in physics-informed neural networks; evaluating LLM training data at Amazon. BS, Computer Engineering; MS, Software Development (2027).

EDUCATION

Boston University | MS in Software Development 2025 - 2027

University of Maine | BS in Computer Engineering, Minor: Computer Science 2021 - 2025

TECHNICAL SKILLS

Languages: Python, TypeScript, JavaScript, Java, C/C++

Frameworks & Libraries: React, FastAPI, Node.js/Express, Flask, PyTorch, TensorFlow

Data & Infrastructure: PostgreSQL, pgvector, Redis, MongoDB, SQLite, Docker, AWS, Git

AI / ML: Retrieval-augmented generation (RAG), vector search (HNSW), ONNX Runtime, LightGBM, LLM evaluation, PINNs

WORK EXPERIENCE

ML Data Associate II | Amazon **July 2025 - Present**

- Design adversarial prompts to surface LLM failure modes; evaluate and label 35,000+ multimodal training samples at ~99% quality and ~2.25× standard throughput, resolving ambiguous or underspecified guidelines.

ML Research Assistant | Advanced Structures and Composites Center **May 2023 - September 2024**

- Designed a multi-fidelity active-learning system in PyTorch (physics-informed and feedforward neural networks) that replaced 28–100-day high-fidelity experiments with analytical low-fidelity models running in hours to days, generating far more training data than physical runs alone could provide.
- Ported a 3,000-line MATLAB simulation codebase to modular Python, converting a manual process into an automated pipeline that runs unattended inside the active-learning loop.
- One of three technical leads (neural networks, multi-fidelity ML, simulation) on a 10-person interdisciplinary research team; owned neural-network design and implementation.

PROJECTS

The Bullpen | thebullpen.net **May 2026 - Present**

- Built and operate a self-hosted baseball analytics platform serving four ML models (LightGBM pitch heads and a 30-park multi-head MLP) from a Java / Spring Boot API with in-process ONNX Runtime, achieving p99 34 ms at a verified 300 req/s.
- Wrote the ML systems layer from scratch (no MLflow): model registry with feature-schema-hash enforcement, shadow A/B routing, PSI and calibration drift detection, and a retraining queue; model promotion is human-gated behind pre-declared criteria enforced in service code and SQLite triggers.
- Proved retraining end-to-end: a queued trigger retrained the serving model on ~1.2M batted balls (labels from an RK4 drag/Magnus physics simulator) unattended in 96.8 minutes, cutting per-park calibration error (ECE) 10×; the post-pitch head scored 59.1% top-1 / 80.8% top-2 on 237k held-out 2026 pitches.
- Operate with production discipline: 2,400+ tests behind gated coverage floors, CI-enforced temporal-leakage and cross-language ONNX parity checks, 28 Prometheus alerts, drilled backups (measured 25-min restore), and written postmortems, including one that caught a silent two-month staleness failure.

StudyForesight | https://studyforesight.com **March 2026 - May 2026**

- Built and deployed a full-stack RAG study platform (FastAPI/Python, React 19/TypeScript, Postgres + pgvector): an LLM tutor answers questions grounded only in the user's own uploads (PDFs, images via OCR, audio via Whisper, Word docs), with every answer citing its source passages.
- Engineered a two-stage retrieval pipeline: pgvector cosine search over an HNSW index, then Maximal Marginal Relevance reranking for diversity, fronted by a Redis semantic cache that answers near-duplicate questions without an LLM call, cutting repeat-query latency and inference cost.
- Designed a fault-tolerant ingestion pipeline: dual execution paths (FastAPI background tasks plus a durable QStash queue) made idempotent via atomic Redis SETNX, with tenant isolation and abuse protection from Postgres row-level security, per-provider circuit breakers, and Redis sliding-window rate limiting.
- Hardened against prompt injection (separated system/user prompt construction, model-echo stripping) and built a RAG-quality evaluation harness with synthetic load testing to catch regressions.